Universidad de Carabobo
Facultad Experimental de Ciencias y Tecnología
Departamento de Computación

Proyecto Final de Arquitectura del Computador

(Optimización de Bloques de Caché con Prefetching)

Profesor:
José Canache
Arquitectura del Computador

Autor:
Ronald Montilla
C.I. 32.011.453

Octubre 2026

Índice

1. El Problema **2**

2. Técnicas Computacionales para su Solución **3**
2.1. El motor de prefetching 3
2.1.1. Prefetching Secuencial 3
2.1.2. Prefetching por Saltos 3
2.2. La Memoria Cache 4

3. Análisis y Comparación de Resultados **6**
3.1. El dominio absoluto del Prefetching Secuencial 6
3.2. El Prefetch de Salto y la Polución de Caché 7
3.3. Conclusión de las pruebas 7

1. El Problema

La memoria cache fue inventada en los años 60s como una solución al cada vez mayor problema del "cuello de botella" que producían las altas velocidades de los procesadores, ya que esta velocidad aumentaba constantemente pero la velocidad de la Memoria RAM no lo hacía, debido a que los científicos se enfocaban en aumentar su capacidad y no su velocidad. Esto generaba el ya mencionado "cuello de botella": Los procesadores eran veloces pero debían esperar mucho tiempo inactivo esperando la respuesta de la memoria RAM.

La memoria cache fue una solución a esto, pero con el tiempo los científicos notaron que a pesar de las altas velocidades de la memoria cache y la CPU, había un vacío de eficiencia y productividad al arranque del hardware: los primeros datos que pide la CPU siempre son fallos porque la cache inicia vacía, esto requiere que la CPU este inactiva durante muchos ciclos de reloj, mientras la RAM transmite los datos solicitados a la cache y esta última a la CPU. Pero no solo en el arranque del hardware se produce este problema de productividad con la CPU, sino durante todo el transcurso del programa mientras la CPU procesa millones de datos y la memoria cache esta en constante trabajo, cargando y vaciando datos en su memoria.

Es aquí donde surge el problema que aborda el presente proyecto, ¿Como hacer que la CPU no este inactiva durante los periodos de tiempo que producen los fallos al arranque de la CPU y durante toda la ejecución del programa? Pues, esto tiene una solución y es la "captación previa", mejor conocido como prefetching. El Prefetching es un sistema que va integrado en el hardware y que permite analizar los patrones de acceso a datos de la memoria, para predecir que datos podría solicitar la CPU y cargarlos con anticipación en la memoria cache. Esto permite mejorar la tasa acierto y no tener una CPU inactiva, y poco productiva, durante el transcurso del programa, ya que el procesador se encontrara analizando los patrones de acceso y cargado datos a la memoria cache con anticipación.

Pero el prefetching no es una solución totalmente eficiente, en ocasiones se produce la llamada *polución de cache*, que se produce cuando el sistema de prefetching carga datos en la memoria cache que el procesador en realidad no iba a usar, desperdiciando de esta manera, ciclos de reloj y degradando la eficiencia.Es por eso que el presente proyecto se basa en implementar estas técnicas de prefetching para mejorar el rendimiento en accesos a memoria a través de un simulador creado en C++17 que emula una memoria cache con prefetch basados en patrones de acierto, con el fin de analizar su mejoría, o no, en la tasa de aciertos de la memoria cache.

2. Técnicas Computacionales para su Solución

El presente proyecto se basó en la creación en `C++17` de un simulador de memoria cache con motor de prefetch para analizar su comportamiento al trabajar directamente con los datos. El simulador posee tres partes funcionales, la primera parte es la memoria cache, la segunda parte es el motor de prefetch seleccionable y la tercera y última parte es el controlador que emplea las dos partes anteriores para mostrar los resultados obtenidos.

2.1. El motor de prefetching

Mediante el uso de la programación orientada a objetos(POO) se creó una clase base llamada `class Prefetch` que representa a motor general de prefetching, esta clase se empleó mediante el uso del polimorfismo para el manejo dos clases hijas que representarían los dos modelos de prefetch empleados en este proyecto, el prefetch secuencial y el prefetch por salto.—

2.1.1. Prefetching Secuencial

El prefetching secuencial se basa en la anticipación de datos que se encuentran consecutivos, básicamente este motor asume que se procesarán datos consecutivos y realiza precarga de datos basado en ello. Para este motor se empleó la técnica de Flujo de Buferes(Stream Buffers), en donde, al producirse un fallo de cache el algoritmo reserva espacio de uno de sus buferes mediante la técnica de LRU(Least Recently Used) para alojar los k-esimos bloques consecutivos. De esta manera, si se cumple el principio de localidad espacial, los datos que podría necesitar el procesador ya estarán cargados en estos búferes.

se empleó la clase hija PrefetchSec para realizar el trabajo del motor de prefetch secuencial. Para simular su estructura se creó un vector que contiene los búferes y se empleó el uso de funciones polimórficas para aplicar su lógica principal, y es que, ante un fallo de cache este algoritmo buscara el dato en su búfer, en caso de encontrarlo el procesador podrá emplear este dato directamente desde el búfer, produciendo un *hit*, pero si no lo encuentra el algoritmo tomará los k-esimos bloques consecutivos al bloque del miss y se seguirá considerando como un miss.

Cada búfer es representado por una estructura que contiene los datos del búfer, como el contandor(contLRU) que permitirá saber si el búfer fue el más reciente usado o no, la cantidad de elementos que contiene el búfer(numElem), el bit de validez(validez) y las lineas que contiene a partir de un vector `std::vector<lineaBuffer> lineas`.

2.1.2. Prefetching por Saltos

El prefetching por salto es más inteligente que el secuencial, pero es empleado para un comportamiento diferente. En primer lugar, el prefetching por salto analiza el comportamiento de las peticiones de datos con el fin de identificar un patrón de salto, confirmar ese patrón y luego ejecutar la precarga. Para llevar a cabo lo antes mencionado, el prefetching por salto emplea una Tabla de Referencia de Predicción, esta tabla toma cada instrucción y verifica sus accesos a datos bajo un modelo específico, es decir, es una tabla que posee n-líneas y por cada línea habrá una instrucción que produjo un fallo de cache, y que sera estudiada con el fin de establecer su patrón de accesos a datos, la línea empezará en estado inicial(initial), cuando dicha instrucción vuelva a pedir un dato se establecerá

su rango de salto y se guardará como *ultimo salto(last stride)*, pasando de esta manera a estado transitorio(transient), por ultimo, si la instrucción pide un dato y este cumple con el rango de salto establecido en el ultimo salto, entonces esa linea de la tabla RPT pasa a esto de confianza(confident) trayendo el próximo bloque a la cache que cumple con el salto, que también puede ser negativo, es decir, un salto hacia atrás.

Para esta implementación se creó una clase hija llamada PrefetchSal y que a través de una estructura se representa la Tabla de Referencia de Predicción, en donde cada línea guardará la última dirección, el ultimo salto, el estado, el contador para saber cual fue la línea recientemente usada, y un bit de validez.

```
struct referencePT {
    uint32_t tag, lastAddress, cont = 0;
    int32_t lastStride : 30;
    uint32_t confidence : 2;
    bool validez = false;
};

class PrefetchSal : public Prefetch {
    private:
        uint32_t numLineas;
        std::vector<referencePT> tablaRPT; // Tabla de prediccion
    de referencia;
    public:
        PrefetchSal(uint32_t _numLineas, uint32_t _tamB);
        void actualizarCont(uint32_t linea);
        uint32_t buscarLineaLRU();
        bool logicaPrefetch(uint32_t pc, uint32_t dir) override;
        void cargarLinea(uint32_t pc, uint32_t dir);
        bool notificarTipo() override;
        void notificarAccion(uint32_t pc, uint32_t *contenedor)
   override;
        ~PrefetchSal();
};
```

Su función principal y polimórfica `bool logicaPrefetch(uint32_t pc, uint32_t dir) override` permitirá realizar la lógica principal y más importante de este motor de Prefetch, y que retornara *verdadero* unicamente cuando un linea pase a estado *confident* e indique que el próximo bloque deberá ser cargado en la memoria cache para intentar predecir su uso.

2.2. La Memoria Cache

Para garantizar mejores y variados resultados con diferentes tipos de datos, se decidió implementar una *Memoria Cache Asociativa por Conjuntos de N-Vías*, que es el punto intermedio entre una memoria cache con mapeo directo y una totalmente asociativa. La estructura de esta memoria cache en código es la siguiente:

Una estructura, `struct linea`, permite simular las vías de cada conjunto de la memoria cache bajo los campos *"tag"* que permite guardar la etiqueta de una dirección, *"cont"* que permite saber cual fue la vía menos y más usada, y por ultimo, el campo *"bitValidez"* para saber si esa vía es útil y actual. Ahora bien, para los conjuntos se empleó otra estructura

con un solo campo, un vector de tipo linea. Utilizando los contenedores de la *Standard Template Library* (STL) y aplicando diseño orientado a objetos se creó la clase base, `class Cache`, cuyos principales datos(privados) son un vector de tipo conjunto para simular la estructura interna de la cache y guardar los datos, las características de la cache(como el tamaño de los bloques, la cantidad de conjuntos y la cantidad de vías), las características de las direcciones(cuantos son los bits de offset, los bits de conjuntos y los bits de tag), y el dato más importante de clase es un apuntador a una clase *Prefetch*.

La lógica principal de la clase *Cache* está en su función `bool buscarDato()`, que recibe el contador de programa de una instrucción y una dirección, y cuya lógica es buscar la dirección en su estructura de conjuntos y vías y retornar verdadero(un hit) en caso de encontrarlo o falso(un miss) en el caso contrario. Además, mediante el uso de *Polimorfismo* se emplea el apuntador al motor de Prefetch para aplicar su lógica de predicciones.

Finalmente, para cumplir con el requisito de *Selección del Tamaño de Bloque* y evaluar el impacto de la geometría de la memoria, se desarrolló un sistema de configuración dinámica en el controlador principal. Este sistema permite al usuario definir en tiempo de ejecución el tamaño de bloque, la cantidad de vías y los conjuntos. Para garantizar la integridad estructural y evitar colapsos matemáticos en las operaciones a nivel de bits, se implementó un algoritmo de validación utilizando la expresión booleana `valor & (valor - 1) == 0`. Esta técnica computacional asegura estrictamente que todas las dimensiones ingresadas por el usuario sean potencias de dos, requisito indispensable para el correcto direccionamiento del hardware simulado.

En resumen, para todos los algoritmos antes mencionados se emplearon varias técnicas computacionales y herramientas que nos brinda el lenguaje de programación C++17, empezando con el uso de la programación orientada a objetos(POO), el polimorfismo y la herencia para crear las estructuras tanto de la memoria cache como de los motores de prefetching. También se emplearon librerías muy importantes de la STL(Standard Template Library) como `<vector>` para crear vectores dinámicos, también la librería `<cstdint>`, para emplear los tipos de datos `uint32_t y int32_t`, y garantizar la mayor portabilidad y semejanza con el hardware real de direcciones de 32bits. De igual manera, para asegurar un funcionamiento idéntico a como lo hace el hardware, para la manipulación de las direcciones se emplearon operadores de bits, como `<<, >>, &, |`, para manipular los bits de las direcciones a conveniencia. Por otra parte, se empleó una técnica de reemplazo moderna y confiable como lo es la técnica LRU(Least Recently Used), para reemplazar aquellos elementos(como las lineas de un búfer, la línea de la tabla RPT o una línea de cache) menos usado recientemente y respaldar el principio de localidad temporal.

3. Análisis y Comparación de Resultados

Para evaluar la efectividad de los motores de prefetching implementados, se sometió el simulador a una serie de pruebas utilizando diferentes trazas de memoria. Estas trazas fueron generadas a partir del procesamiento de matrices dispersas (formato Matrix Market), lo que nos permitió evaluar los algoritmos frente a cargas de trabajo con accesos tanto lineales como erráticos. Para medir el éxito de cada motor, el enfoque principal de este análisis no recae únicamente en la Tasa de Acierto general, sino en la **Cobertura (Coverage)**, la cual nos indica exactamente qué porcentaje de los fallos originales de la caché base lograron ser eliminados por la precarga.

Tras analizar los resultados de las múltiples simulaciones (resumidos en el **Cuadro 1**), los datos arrojaron un patrón de comportamiento muy claro y fascinante sobre cómo interactúa el hardware predictivo con las estructuras de datos complejas.

Cuadro 1: Comparación de Rendimiento y Cobertura entre Motores de Prefetching (Resultados Completos)

Escenario de Prueba	Motor de Prefetch	Hits	Misses	Tasa de Acierto	Cobertura (Coverage)
Prueba 1 (Volumen Bajo)	Sin Prefetch (Base)	764.974	14.475	98.14 %	-
	Secuencial	771.375	8.074	98.96 %	**44.22 %**
	De Salto (Stride)	764.949	14.500	98.13 %	**-0.17 %**
Prueba 2 (Volumen Medio-Bajo)	Sin Prefetch (Base)	917.097	14.565	98.43 %	-
	Secuencial	923.582	8.080	99.13 %	**44.52 %**
	De Salto (Stride)	917.070	14.592	98.43 %	**-0.18 %**
Prueba 3 (Volumen Medio)	Sin Prefetch (Base)	1.301.679	14.819	98.87 %	-
	Secuencial	1.308.299	8.199	99.37 %	**44.67 %**
	De Salto (Stride)	1.301.653	14.845	98.87 %	**-0.17 %**
Prueba 4 (Volumen Medio-Alto)	Sin Prefetch (Base)	2.376.655	16.738	99.30 %	-
	Secuencial	2.385.454	7.939	99.66 %	**52.56 %**
	De Salto (Stride)	2.376.634	16.759	99.29 %	**-0.12 %**
Prueba 5 (Patrón Constante)	Sin Prefetch (Base)	2.609.695	49.144	98.15 %	-
	Secuencial	2.618.070	40.769	98.46 %	**17.04 %**
	De Salto (Stride)	2.617.886	40.953	98.45 %	**16.66 %**
Prueba 6 (Volumen Alto)	Sin Prefetch (Base)	3.360.394	17.233	99.48 %	-
	Secuencial	3.369.197	8.430	99.75 %	**51.08 %**
	De Salto (Stride)	3.360.353	17.274	99.48 %	**-0.23 %**
Prueba 7 (Volumen Extremo)	Sin Prefetch (Base)	8.885.150	18.881	99.78 %	-
	Secuencial	8.895.496	8.535	99.90 %	**54.79 %**
	De Salto (Stride)	8.885.122	18.909	99.78 %	**-0.14 %**

3.1. El dominio absoluto del Prefetching Secuencial

Como era de esperarse al recorrer arreglos y matrices en `C++`, el prefetching secuencial obtuvo resultados sobresalientes en la inmensa mayoría de las pruebas. Al observar los datos, este motor logró coberturas sumamente altas, rondando entre el 44 % y casi el 55 % de fallos reducidos.

Por ejemplo, en una de las pruebas de mayor volumen, la caché sin prefetch registró 18,881 fallos. Al activar el motor secuencial, los fallos se desplomaron a tan solo 8,535, logrando una cobertura del 54.79 %. Este mismo patrón de éxito se repitió en otras ejecuciones, alcanzando coberturas del 52.56 % y 51.08 %.

Este rendimiento se debe a que las matrices dispersas se leen linealmente de principio a fin al momento de cargar sus coordenadas. El motor secuencial aprovechó al máximo el

principio de localidad espacial: al detectar un fallo, sus búferes de flujo (*Stream Buffers*) se adelantaban a la CPU descargando los bloques contiguos, lo que resultó ser una estrategia sumamente agresiva y efectiva para ocultar la latencia en este tipo de lectura.

3.2. El Prefetch de Salto y la Polución de Caché

Lo más interesante de estos resultados fue el comportamiento del motor de prefetching por salto (Stride). En casi todas las pruebas ejecutadas con las matrices dispersas, el motor de salto no solo falló en mejorar el rendimiento, sino que **empeoró** el estado de la caché, arrojando una cobertura negativa.

Los datos son contundentes: en varias simulaciones, el motor de salto registró coberturas de -0.18 %, -0.23 % y -0.17 %. Esto significa que causó más fallos que la caché base. Un ejemplo claro se ve en una de las pruebas donde la caché original tuvo 14,565 fallos, y al encender el prefetch de salto, los fallos aumentaron a 14,592.

¿Por qué ocurrió esto? Esto es la demostración empírica de un fenómeno llamado la **Polución de Caché**. Al realizar la multiplicación de la matriz dispersa contra un vector, el procesador debe saltar de forma caótica por la memoria dependiendo de los índices de las columnas no nulas. La máquina de estados del motor de salto (su tabla RPT) intentó buscar desesperadamente un patrón matemático constante en estos saltos. Al creer erróneamente que había encontrado un patrón (estado *Confident*), el motor comenzó a inyectar bloques inútiles a la memoria caché, expulsando datos que la CPU sí iba a necesitar, lo que incrementó artificialmente la tasa de fallos.

3.3. Conclusión de las pruebas

Cabe destacar que el prefetch de salto no es un mal algoritmo. De hecho, en una de las trazas de prueba, creadas específicamente para probrar el prefetcing de salto, logró estabilizarse, alcanzando una cobertura positiva del 16.66 % (muy a la par del 17.04 % que logró el secuencial en esa misma prueba). Esto demuestra que cuando el motor logra encontrar una constante en los saltos de memoria, hace bien su trabajo.

Sin embargo, el análisis general de este proyecto nos deja una lección: el prefetching es un arma de doble filo. Mientras que el prefetch secuencial demostró ser una optimización robusta y segura para cargas de trabajo estándar, el prefetch por salto requiere escenarios de memoria muy predecibles. Si se aplica un prefetcher agresivo a un patrón de datos altamente irregular, el hardware terminará saboteando su propio rendimiento.